

que contém um ponteiro para função para a versão da função `virtual` a ser usada em um objeto dessa classe. A função `virtual` a ser usada em determinada classe poderia ser a função definida nessa classe, ou poderia ser uma função herdada direta ou indiretamente de uma classe-base mais alta na hierarquia.

- Quando uma classe-base fornece uma função-membro `virtual`, classes derivadas podem sobrepor a função `virtual`, mas elas não precisam sobrepor-la.
- Cada objeto de uma classe com funções `virtuais` contém um ponteiro para a `vtable` dessa classe. Quando uma chamada de função é feita a partir de um ponteiro da classe-base para um objeto da classe derivada, o ponteiro para função apropriado na `vtable` é obtido e desreferenciado para completar a chamada no tempo de execução.
- Qualquer classe que tenha um ou mais ponteiros 0 em sua `vtable` é uma classe abstrata. As classes sem quaisquer ponteiros 0 na `vtable` são classes concretas.
- Novos tipos de classes são regularmente acrescentados aos sistemas e acomodados pela vinculação dinâmica.

Seção 21.8 Estudo de caso: sistema de folha de pagamento usando polimorfismo e informação de tipo em tempo de execução com `downcasting`, `dynamic_cast`, `typeid` e `type_info`

- O operador `dynamic_cast` verifica o tipo do objeto ao qual um ponteiro aponta, depois determina se o tipo tem um re-

lacionamento *é um* com o tipo ao qual o ponteiro está sendo convertido. Se tiver, `dynamic_cast` retorna o endereço do objeto. Se não, `dynamic_cast` retorna 0.

- O operador `typeid` retorna uma referência a um objeto `type_info` que contém informações sobre o tipo do operando, incluindo o nome do tipo. Para usar `typeid`, o programa precisa incluir o arquivo de cabeçalho `<typeinfo>`.
- Quando chamada, a função-membro `name` do objeto `type_info` retorna uma string baseada em ponteiro que contém o nome do tipo que o objeto `type_info` representa.
- Os operadores `dynamic_cast` e `typeid` fazem parte do recurso de informação de tipo no tempo de execução (RTTI) de C++, que permite que um programa determine o tipo do objeto no tempo de execução.

Seção 21.9 Destrutores virtuais

- Declare o destrutor da classe-base `virtual` se a classe tiver funções virtuais. Isso torna todos os destrutores da classe derivada virtuais, embora eles não tenham o mesmo nome do destrutor da classe-base. Se um objeto na hierarquia for destruído explicitamente aplicando-se o operador `delete` a um ponteiro da classe-base para um objeto da classe derivada, o destrutor para a classe específica será chamado. Após a execução de um destrutor da classe derivada, os destrutores de todas as classes-base dessa classe serão executados até o topo da hierarquia.

Terminologia

classe de iteração 678

classes abstratas 676

classes concretas 676

classes-base abstratas 676

conversão dinâmica 660

destrutores não virtuais 699

`downcasting` 669

`dynamic_cast` 698

especificador puro (com funções virtuais) 676

função `name` da classe `type_info` 698

função `virtual` pura 676

herança de implementação 679

herança de interface 679

informação de tipo em tempo de execução (RTTI) 660

polimorfismo 659

sobreposição 670

`type_info`, classe 698

`typeid`, operador 698

`<typeinfo>`, arquivo de cabeçalho 698

vinculação dinâmica 671

vinculação estática 671

vinculação tardia 671

`virtual`, destrutor 699

`virtual`, função 670

`virtual`, tabela de função (`vtable`) 693

Exercícios de autorrevisão

21.1 Preencha os espaços em cada uma das sentenças:

- Tratar um objeto da classe-base como um(a) _____ pode causar erros.
- O polimorfismo ajuda a eliminar a lógica de _____.

- Se uma classe contém pelo menos uma função `virtual` pura, ela é uma classe _____.
- As classes das quais os objetos podem ser instanciados são chamadas de classes _____.

- e) O operador `dynamic_cast` pode ser usado para fazer o downcast dos ponteiros da classe-base com segurança.
- f) O operador `typeid` retorna uma referência a um objeto `const std::type_info`.
- g) `virtual` envolve o uso de um ponteiro da classe-base ou uma referência para chamar funções virtuais nos objetos da classe-base e da classe derivada.
- h) Funções que podem ser sobrepostas são declaradas usando-se a palavra-chave `virtual`.
- i) A conversão de um ponteiro da classe-base para um ponteiro da classe derivada é chamada de `downcasting`.

21.2 Responda se cada uma das sentenças a seguir é *verdadeira* ou *falsa*. Em caso de alternativas *falsas*, justifique sua resposta.

- a) Todas as funções virtuais em uma classe-base abstrata devem ser declaradas como funções virtuais puras.
- b) Referir-se a um objeto da classe derivada com um handle da classe-base é perigoso.
- c) Uma classe se torna abstrata ao ser declarada `virtual`.
- d) Se uma classe-base declara uma função `virtual`, uma classe derivada deve implementar essa função para se tornar uma classe concreta.
- e) A programação polimórfica pode eliminar a necessidade da lógica de `switch`.

■ Respostas dos exercícios de autorrevisão

- 21.1** a) objeto da classe derivada. b) `switch`. c) abstrata. d) concretas. e) `dynamic_cast`. f) `type_info`. g) Polimorfismo. h) `virtual`. i) `downcasting`.
- 21.2** a) Falso. Uma classe-base abstrata pode incluir funções virtuais com implementações. b) Falso. Referir-se a um

objeto da classe-base com um handle da classe derivada é perigoso. c) Falso. As classes nunca são declaradas como virtuais. Em vez disso, uma classe se torna abstrata ao se incluir pelo menos uma função `virtual` pura na classe. d) Verdadeiro. e) Verdadeiro.

■ Exercícios

- 21.3** Como o polimorfismo permite que você trabalhe uma programação 'geral' em vez de uma 'específica'? Discuta as principais vantagens da programação 'geral'.
- 21.4** Discuta os problemas de programação com a lógica de `switch`. Explique por que o polimorfismo pode ser uma alternativa eficaz ao uso da lógica de `switch`.
- 21.5** Faça a distinção entre herdar interface e herdar implementação. Como as hierarquias de herança projetadas para herdar a interface diferem daquelas projetadas para herdar a implementação?
- 21.6** O que são funções virtuais? Descreva uma circunstância em que funções virtuais seriam apropriadas.
- 21.7** Faça a distinção entre vinculação estática e vinculação dinâmica. Explique o uso de funções virtuais e da `vtable` na vinculação dinâmica.
- 21.8** Faça a distinção entre funções virtuais e funções virtuais puras.
- 21.9** **Classes-base abstratas.** Sugira um ou mais níveis de classes-base abstratas para a hierarquia Forma discutida neste capítulo e exibida na Figura 20.3. (O primeiro nível é Forma, e o segundo nível consiste nas classes FormaBidimensional e FormaTridimensional.)

21.10 Como o polimorfismo promove a extensibilidade?

21.11 Você foi o escolhido para desenvolver um simulador de voo que terá saídas gráficas elaboradas. Explique por que a programação polimórfica poderia ser especialmente eficaz para um problema dessa natureza.

21.12 **Modificação do sistema de folha de pagamento.** Modifique o sistema de folha pagamento das figuras 21.13 a 21.23 com o objetivo de incluir o dado-membro `private data Nascimento` na classe `Funcionario`. Use a classe `Date` das figuras 19.9 e 19.10 para representar a data de nascimento de um funcionário. Suponha que a folha de pagamento seja processada uma vez por mês. Crie um vector de referências a `Funcionario` para armazenar os diversos objetos de funcionário. Em um loop, calcule a folha de pagamento de cada `Funcionario` (polimorficamente), e adicione um bônus de R\$ 100,00 ao valor da folha de pagamento se o mês atual for o do aniversário do `Funcionario`.

21.13 **Hierarquia Forma.** Implemente a hierarquia Forma projetada no Exercício 20.7 (que é baseada na hierarquia da Figura 20.3). Cada `FormaBidimensional` deverá conter a função `retArea` para calcular a área da forma bidimensional. Cada `FormaTridimensional` deverá ter

funções-membro `retArea` e `retVolume` para calcular a área e o volume, respectivamente, da forma tridimensional. Crie um programa que use um vetor de ponteiros `Forma` para objetos de cada classe concreta na hierarquia. O programa deverá imprimir o objeto ao qual cada elemento vetor aponta. Além disso, no loop que processa todas as formas no vetor, determine se cada forma é uma `FormaBidimensional` ou uma `FormaTridimensional`. Se a forma for uma `FormaBidimensional`, mostre sua área. Se a forma for uma `FormaTridimensional`, mostre sua área e seu volume.

21.14 Projeto: gerenciador de tela polimórfico usando a hierarquia *Forma*. Desenvolva um pacote gráfico básico. Use a hierarquia `Forma` implementada no Exercício 21.13. Limite-se a formas bidimensionais, como quadrados, retângulos, triângulos e círculos. Interaja com o usuário. Permita que o usuário especifique a posição, o tamanho, a forma e os caracteres de preenchimento a serem usados no desenho de cada forma. O usuário pode especificar mais de uma da mesma forma. Ao criar cada forma, coloque um ponteiro `Forma *` para cada novo objeto `Forma` em um array. Cada classe `Forma` agora deverá ter sua própria função-membro `desenhar`. Escreva um gerenciador de tela polimórfico que atravesse o array, enviando mensagens `desenhar` a cada objeto no array para formar uma imagem na tela. Redesenhe a imagem da tela toda vez que o usuário especificar uma forma adicional.

21.15 Hierarquia de herança *Pacote*. Use a hierarquia de herança `Pacote` criada no Exercício 20.9 para criar um

programa que mostre a informação de endereço e calcule os custos de remessa para vários `Pacotes`. O programa deverá conter um vetor de ponteiros `Pacote` para objetos das classes `PacoteDoisDias` e `PacoteNoturno`. Percorra o vetor para processar os `Pacotes` polimorficamente. Para cada `Pacote`, chame funções `get` para obter a informação de endereço do emissor e do destinatário, depois imprima os dois endereços conforme apareceriam em etiquetas de correspondência. Além disso, chame a função-membro `calculaCusto` de cada `Pacote` e imprima o resultado. Acompanhe o custo de entrega total para todos os `Pacotes` no vetor e mostre esse total quando o loop terminar.

21.16 Programa bancário polimórfico usando a hierarquia *Conta*. Desenvolva um programa bancário polimórfico usando a hierarquia `Conta` criada no Exercício 20.10. Crie um vetor de ponteiros `Conta` para objetos `ContaPoupança` e `ContaCorrente`. Para cada `Conta` no vetor, permita que o usuário especifique um valor a ser sacado da `Conta` usando a função-membro `debitar` e um valor a ser depositado na `Conta` usando a função-membro `creditar`. Ao processar cada `Conta`, determine seu tipo. Se uma `Conta` for uma `ContaPoupança`, calcule o valor dos juros devidos à `Conta` usando a função-membro `calculaJuros`, depois some os juros ao saldo da conta usando a função-membro `creditar`. Depois de processar uma `Conta`, imprima o saldo atualizado da conta, obtido chamando a função-membro da classe-base `retSaldo`.

Fazendo a diferença

21.17 Classe abstrata *EmissaoCarbono*: polimorfismo.

Usando uma classe abstrata com apenas funções virtuais puras, você pode especificar comportamentos semelhantes de classes possivelmente divergentes. Governos e empresas do mundo inteiro estão cada vez mais preocupadas com as emissões de carbono (liberações anuais de dióxido de carbono na atmosfera) de prédios que consomem vários tipos de combustíveis para obter calor, veículos que consomem combustíveis para obter potência, e coisas desse tipo. Muitos cientistas culpam esses gases do efeito estufa pelo fenômeno chamado aquecimento global. Crie três classes pequenas não relacionadas por herança — classes `Predio`, `Carro` e `Bicicleta`. Dê a cada classe alguns atributos e comportamentos apropriados exclusivos que

ela não tenha em comum com as outras classes. Escreva uma classe abstrata `EmissaoCarbono` com apenas um método virtual puro `retEmissaoCarbono`. Faça com que cada uma de suas classes herde dessa classe abstrata e implemente o método `retEmissaoCarbono` para calcular uma emissão de carbono apropriada para essa classe (verifique em alguns sites que explicam como calcular a emissão de carbono). Escreva uma aplicação que crie objetos de cada uma das três classes, coloque ponteiros para esses objetos em um vetor de ponteiros `EmissaoCarbono` e depois percorra o vetor, chamando polimorficamente o método `retEmissaoCarbono` de cada objeto. Para cada objeto, imprima informações de identificação e sua emissão de carbono.